Key Design Decisions

A. How alarm consequences are dispatched to different parts of the system

Design 1: AlarmFloor directly calls methods on Undead and spawns drones itself when a worker steps on it.

Pros	Cons
Simple and direct; all alarm logic is in one place.	AlarmFloor becomes tightly coupled to every consequence class (Undead, SecurityDrone), violating the Single Responsibility Principle.
	Adding a new consequence (e.g., locking doors) requires modifying AlarmFloor, violating the Open-Closed Principle.
	AlarmFloor depends on concrete consequence classes rather than abstractions (violates Dependency Inversion Principle).

Design 2: Use the Observer pattern. AlarmManager (singleton subject) maintains a list of AlarmListener observers. AlarmFloor triggers the alarm via AlarmManager, which notifies all registered listeners. Each listener defines its own response independently.

Pros	Cons
AlarmFloor depends only on AlarmManager, not on any consequence class (Dependency Inversion Principle). It does not know what happens when the alarm fires.	Requires an additional manager class and interface, adding structural complexity.
New consequences can be added by implementing AlarmListener and registering with the manager — no existing code needs to change (Open-Closed Principle).	The indirection through a singleton may make the control flow less immediately visible.
Each listener handles exactly one consequence (Single Responsibility Principle): Undead handles its own behaviour change; the drone-spawning lambda handles drone creation.	
The Observer pattern is a well-established design pattern that cleanly separates event production from event handling.	

B. How the Undead responds to the alarm (hunting workers)

Design 1: Undead.playTurn() checks a global alarm flag each turn and, if active, manually computes movement toward the nearest worker.

Pros	Cons
No dependency on external notification; self-contained check.	Duplicates pathfinding logic that FollowBehaviour already provides, violating DRY.
	Mixes alarm-response logic with normal turn logic, violating Single Responsibility.
	Every Undead polls the flag every turn, even when the alarm has not been triggered.

Design 2: Undead implements AlarmListener. When notified, it injects a FollowBehaviour targeting the triggering worker into its behaviour list at position 1 (between attack and wander). The existing behaviour iteration in playTurn() handles the rest.

Pros	Cons
Reuses the existing FollowBehaviour and the behaviour-list iteration pattern from REQ3, avoiding code duplication (DRY).	Undead must register with AlarmManager in its constructor, creating a dependency on the singleton.
The alarm response is a one-time mutation (inserting a behaviour), not a per-turn check — no polling overhead.	Dynamically modifying the behaviour list at runtime could be surprising if not documented.
playTurn() remains unchanged — the priority system (attack > follow > wander) naturally handles the new behaviour without any conditional logic.	
Follows the Open-Closed Principle: the alarm response is added without modifying the existing playTurn() method or any other behaviour class.	

C. How Security Drones are spawned (drone consequence)

Design 1: Create a dedicated DroneSpawner class that listens for the alarm and handles spawning.

Pros	Cons
Dedicated class with a clear single responsibility.	Adds an extra class for what is essentially a few lines of spawning logic.

Design 2: Register a lambda AlarmListener in EclipseNebula.initialise() that spawns SecurityDrone actors at predefined locations when the alarm triggers.

Pros	Cons
Keeps spawning logic co-located with other map initialisation code, which is where spawn points are defined.	The lambda is less visible than a named class; could be missed when reading the codebase.
Avoids an extra class for a simple, map-specific operation.	If spawning logic grows more complex, it may need to be extracted into its own class.
Still satisfies Single Responsibility: EclipseNebula is responsible for world setup, and registering alarm listeners is part of that setup.	

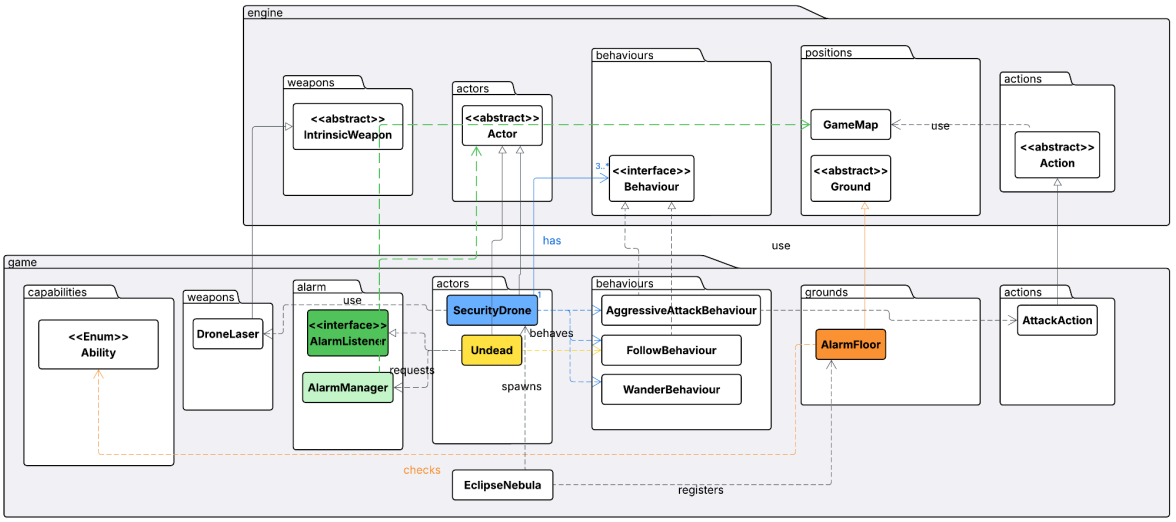
D. How Security Drones select targets (aggressive vs. selective attack)

Design 1: Reuse the existing AttackBehaviour from REQ3, which checks Ability.ATTACKABLE_BY_CREATURES.

Pros	Cons
No new class needed; reuses existing code.	Drones would only attack workers, not other creatures. The specification states drones attack "any moving entity (workers and other creatures alike)."
	Changing AttackBehaviour to remove the ability check would break Undead's selective targeting. Modifying a shared class to serve one consumer violates the Liskov Substitution Principle.

Design 2: Create a separate AggressiveAttackBehaviour that attacks the first adjacent actor found, without filtering by ability.

Pros	Cons
Correctly implements the specification: drones attack all adjacent entities indiscriminately.	Adds a new behaviour class that is structurally similar to AttackBehaviour.
AttackBehaviour (selective) and AggressiveAttackBehaviour (indiscriminate) can coexist, each serving different creature types without interference (Open-Closed Principle).	
Both implement the engine's Behaviour<Actor, Action> interface, maintaining consistency with the Strategy pattern established in REQ3.	



Summary

The final design of requirement 4 utilises Design 2 of parts A, B, C, and D stated above to implement the facility alarm system, Undead hunting response, drone spawning, and drone targeting.

The alarm system is built around the Observer pattern. AlarmManager is a singleton subject that maintains a list of AlarmListener observers. AlarmFloor detects workers via Ability.ATTACKABLE_BY_CREATURES (the same ability enum used for Undead targeting in REQ3) and calls AlarmManager.triggerAlarm(). The manager then notifies all registered listeners, each of which handles its own consequence independently. This means AlarmFloor has no knowledge of what the alarm does — it only knows how to trigger it — satisfying the Dependency Inversion Principle and Single Responsibility Principle. New consequences can be added by registering additional listeners, with no changes to AlarmFloor or AlarmManager (Open-Closed Principle).

The Undead's alarm response is implemented via the AlarmListener interface. When notified, the Undead injects a FollowBehaviour (targeting the triggering worker) into its behaviour list between attack and wander. This reuses the existing FollowBehaviour class and the behaviour-iteration pattern from REQ3, so playTurn() requires no modification. The priority ordering (attack > follow > wander) naturally ensures the Undead attacks when adjacent and follows when not.

Security Drones are spawned by a lambda AlarmListener registered in EclipseNebula.initialise(). Each drone is initialised with an AggressiveAttackBehaviour (attacks any adjacent entity), a FollowBehaviour (pursues the triggering worker), and a WanderBehaviour (fallback). The AggressiveAttackBehaviour is distinct from AttackBehaviour because drones target all entities indiscriminately, while Undead selectively target workers — sharing a single behaviour class would violate the Liskov Substitution Principle by forcing one class to serve two incompatible contracts.

All new classes follow the existing patterns established in earlier requirements: behaviours implement the engine's Behaviour<Actor, Action> interface, weapons extend IntrinsicWeapon, and abilities are mediated through the Ability enum. This consistency

ensures that the alarm system integrates cleanly with the existing architecture rather than introducing ad-hoc mechanisms.